

INDIVIDUAL MODULE-WISE LAB WORKBOOK

EXPERIMENT 2

Module: Unit Test Frameworks

Student Name: Puspendu Sekhar Das

Enrollment Number: 12023002022220

Department: CST

Institution: Institute of Engineering and Management, Newtown

Experiment: Unit Test Frameworks

Module Name and Experiment Title

Module: Unit Test Frameworks

Title: Unit Testing and Test Automation Using Unittest, PyTest and Page Object Model

Problem Statement

Manual execution and verification of individual program components becomes difficult as the number of test cases increases. Python unit-test frameworks provide structured mechanisms for creating, organizing and executing repeatable tests. The experiment demonstrates unittest and PyTest concepts including test discovery, setup/teardown, assertions, fixtures, parameterization, command-line execution, reporting and a basic Page Object Model structure.

Objective

- Understand the purpose and structure of Python unit testing frameworks.
- Create and execute test cases using unittest and PyTest conventions.
- Use assertions, setup/teardown methods and PyTest fixtures.
- Use conftest.py, parameterization, test selection and command-line options.
- Understand skips, expected failures, parallel execution and HTML reporting.
- Organize Selenium automation using the Page Object Model and reusable utility components.

Tools, Software and Concepts Used

Category	Details
Frameworks	Python unittest and PyTest
Programming Language	Python 3
IDE / Terminal	Visual Studio Code / PyCharm / Windows PowerShell
Test Organization	Test files, test methods, suites and discovery conventions
PyTest Features	Fixtures, conftest.py, parameterization, markers, skips and expected failures
Framework Design	Page Object Model, base page, page classes, test classes and utilities
Reporting	PyTest HTML reports and execution output

Implementation Steps

- Create a test file using unittest and define test methods with assertions.
- Configure PyTest naming conventions and create test functions using assert statements.
- Use fixtures for reusable setup and teardown and conftest.py for shared configuration.
- Parameterize a test so that multiple input combinations are executed by one test definition.
- Run selected tests from the command line and generate an HTML execution report.
- Separate page locators and browser actions from test assertions using Page Object Model principles.

Source Code / Representative Implementation

```
import unittest

class TestCalculator(unittest.TestCase):

    @classmethod
    def setUpClass(cls):
        print("Class setup")

    def test_addition(self):
        self.assertEqual(2 + 3, 5)

    @classmethod
    def tearDownClass(cls):
        print("Class teardown")

if __name__ == "__main__":
    unittest.main()
```

Unittest Code

```
1 import unittest
2
3 class TestCalculator(unittest.TestCase):
4
5     @classmethod
6     def setUpClass(cls):
7         print("Class setup")
8
9     def test_addition(self):
10         self.assertEqual(2 + 3, 5)
11
12     @classmethod
13     def tearDownClass(cls):
14         print("Class teardown")
15
16 if __name__ == "__main__":
17     unittest.main()
```

Figure 1: Python unittest implementation with class-level setup and teardown

PyTest Implementation

```
import pytest

@pytest.fixture
def setup():
    print("Setup executed")
    yield
    print("Teardown executed")

@pytest.mark.parametrize("a,b,expected", [
    (2, 3, 5),
    (10, 5, 15)
])
def test_addition(a, b, expected, setup):
    assert a + b == expected
```

Pytest Code

```
1 import pytest
2
3 @pytest.fixture
4 def setup():
5     print("Setup executed")
6     yield
7     print("Teardown executed")
8
9 @pytest.mark.parametrize("a,b,expected", [
10     (2, 3, 5),
11     (10, 5, 15)
12 ])
13 def test_addition(a, b, expected, setup):
14     assert a + b == expected
```

Figure 2: PyTest fixture and parameterized test implementation

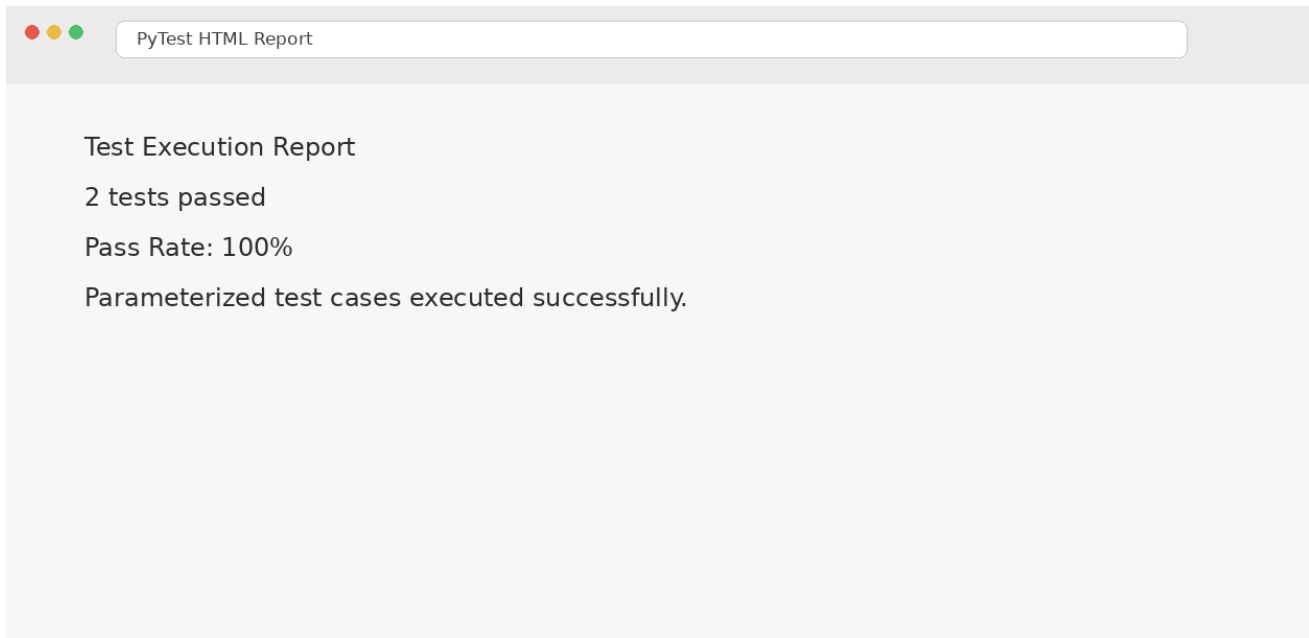
Key Implementation Concepts

- unittest provides a class-based structure with setUp, test methods, assertions and tearDown.
- PyTest discovers files and functions using naming conventions and provides concise assert-based validation.
- Fixtures provide reusable setup and teardown logic; conftest.py can share fixtures across tests.
- Parameterization allows one test definition to execute against multiple data combinations.
- PyTest command-line options can select tests, control execution and support reporting.
- Page Object Model separates page locators and UI methods from test assertions, improving maintainability.
- BasePage and utility classes can centralize common browser and data-handling operations.
- Test failures can be accompanied by screenshots and reports for easier debugging.

Windows PowerShell

```
PS D:\Documents\Wipro Lab Reports> pytest -v
===== test session starts =====
collected 2 items
test_calculator.py::test_addition[2-3-5] PASSED
test_calculator.py::test_addition[10-5-15] PASSED
===== 2 passed in 0.42s =====
```

Figure 3: PyTest execution output from the laboratory terminal



Output with Clearly Labelled Screenshots

Figure 4: HTML-style PyTest execution report

Result

Unit testing and automation were organized using unittest and PyTest concepts. Test methods were executed with assertions, reusable setup/teardown mechanisms and parameterized inputs. The framework structure also supports reporting and Page Object Model based organization.

Observations

- Assertions provide a direct way to compare actual and expected results.
- Fixtures reduce repeated setup and teardown code across test cases.
- Parameterization is useful for executing the same logic with multiple inputs.
- Separating page actions from test assertions improves maintainability.
- Command-line execution and reports provide useful evidence of test status.

Conclusion

The Unit Test Frameworks module was completed through practical demonstrations of unittest, PyTest, fixtures, parameterization, reporting and Page Object Model concepts. These techniques provide a structured approach for creating reusable and maintainable Python test automation.